Exercise 3 (Running Time of Reingold-Tilford Algorithm, 9 Points)

Hugo Lladró Prats, Inés Mang Román, Ahmet Meriç Kızıltaş

The Reingold-Tilford algorithm not only provides tidy tree layouts, but is also computationally efficient. In this exercise, you will consider how the idea of threading that was mentioned in the lecture reduces the time complexity of computing the required horizontal shifts, and how we can reduce the complexity of applying those shifts.

a) *Computing the correct separation between siblings requires traversing the contours of the respective subtrees. If subtrees have heights hL and hR, respectively, what is the complexity of computing the separation by tracing the inner contours? What is the complexity of introducing the new thread (if needed) by additionally tracing the outer contours as well? (2P)*

The complexity of computing the separation is on the order of min(hL, hR), as the algorithm traverses the right contour of the left subtree, which has hL nodes, and the left contour of the right subtree, which has hR nodes, in parallel until one of them runs out of nodes. The complexity of introducing the new thread is also on the order of min(hL, hR), because the algorithm traces the outer contour of the shorter subtree to find the last node, that is, in min(hL, hR) steps, and the inner contour of the longer subtree, until it finds the node at height min(hL, hR) + 1, and creates a pointer from the former to the latter.

b) *Why can any node be visited at most once as part of an inner contour comparison throughout the entire execution of the algorithm? What does this imply with respect to the overall complexity of all separation computations? (2P)*

A node can be visited only once as part of the inner contour because when it is visited while calculating the separation of the subtrees, the means that the other graph also has a node at the same height, and therefore will cease to form part of the inner contour at the end of the iteration. This implies that the complexity of all separation computations in at the order(or less) than the number of nodes in the graph.

c) *Assume that, whenever you place a parent node, you shift all nodes in its subtrees to avoid overlaps, by recursively updating their horizontal coordinate. What is the asymptotic running time for shifting a subtree with ns nodes, and the worst case asymptotic running time for all shifts in the overall algorithm? (2P)*

The asymptotic running time for shifting a subtree with ns nodes would be O(ns), because we have to update the coordinates of all ns-1 sons of the root node. Using this naive algorithm we would have in total quadratic complexity, because in the worst case, for example given by a tree where every left son is a leave, we would have to update the

position of every placed node in every iteration, therefore yielding a complexity in the order of 1+2+...+(ns-1) = ns(ns-1)/2, which is in O(ns^2).

d) *In the implementation proposed by Reingold and Tilford, instead of storing absolute positions, each node initially stores its relative position with respect to its parent as an offset. Describe how one would shift all nodes in a subtree in that case, and state the complexity for a subtree with ns nodes. Also describe in words or as pseudo code how one would assign final absolute positions based on those offsets. What is the time complexity of that process? (3P)*

In this case, when we want to shift an entire subtree, we only need to update the root node. As in each iteration, we only have to shift the root nodes of the 2 subtrees and there are O(ns) iterations, the complexity of the whole process would have O(ns) complexity. When determining the final position of all nodes, we can simply do a Breadth First Search, assingning the position 0 to the root node, and calculating each son's position by getting their parent's position and adding their offset, which would have O(ns) complexity, because every node only has to do a single process. If we want all positions to be non-negative, we can find the minimum x-position and subtract if from every node, both of which can be done in O(ns) time.